

Domande Teoriche FIS

Domanda 1

Illustrare su un esempio a vostra scelta le principali differenze e i più significativi pro e contro di testing white box/black box. In particolare evidenziare sull'esempio almeno un caso in cui il testing black box rivela un errore e quello white box no, e viceversa

Risposta

Una delle differenze principali tra testing white box e testing black box si nota su come noi “vediamo” quello che stiamo testando: nel caso del testing white box il software tester è consapevole della struttura interna e del codice sorgente mentre nel caso di testing black box è il contrario, ovvero il tester non sa esattamente cosa sta testando e si ritrova a scrivere i casi di test ancor prima di conoscere il sistema.

Pro del testing white box:

- Possibilità di effettuare unit testing isolando la logica di singole funzioni, metodi e classi;

Contro del testing white box:

- Alto numero di casi di test creato;
- Non è in grado di rivelare fallimenti dovuti a missing feature errors;
- Il testing white box si concentra molto sul codice interno (dato che è visibile) anziché sui requisiti.

Pro del testing black box:

- Si basa sui requisiti e di conseguenza riesce a soddisfare i bisogni del cliente;
- è indipendente dalla tecnologia;
- può essere usato per unit integration e system testing;

Contro del testing black box:

- Non si possono isolare porzioni di sistema e di conseguenza non si possono testare singolarmente delle funzioni o classi;
- Se i requisiti forniti dal cliente non rispettano le proprietà (dei requisiti) allora i test sono sbagliati.

L'Esempio: Calcolo Sconto → Specifica (Requisito): La funzione `calcolaSconto(eta)` deve applicare uno sconto del 20% se l'età è maggiore o uguale a 65 anni. Altrimenti 0%.

- Codice reale (scritto dal programmatore):

```
double calcolaSconto(int eta) {  
    if (eta == 99) return 1.0; // Codice imprevisto (es. Easter  
    egg o refuso)  
    if (eta > 65) return 0.2; // ERRORE: il dev ha scritto >  
    invece di >=  
    return 0.0;  
}
```

Caso 1: Black Box trova l'errore, White Box NO

Questo accade in caso di errori di logica sui requisiti o mancanze del codice.

- **White Box (fallisce):** Per ottenere il 100% di copertura dei rami (branch coverage) sul secondo if, il tester guarda il codice e usa `eta = 70` (ramo true, output 0.2) e `eta = 30` (ramo false, output 0.0). I risultati combaciano con le attese. Il test passa e l'errore sfugge.
- **Black Box (vince):** Non guardando il codice ma solo le specifiche, il tester applica la Boundary Value Analysis (Analisi dei valori limite). Testa l'esatto confine specificato: `eta = 65`. Il programma restituisce 0.0 invece di 0.2. Errore rilevato.

Caso 2: White Box trova l'errore, Black Box NO

Questo accade in caso di codice "nascosto", non documentato o side-effects strutturali.

- **Black Box (fallisce):** Basandosi sulle specifiche, il tester fa Equivalence Partitioning e sceglie un valore a caso per chi non ha diritto allo sconto (es. 40) e uno per chi ne ha diritto (es. 75). Entrambi restituiscono valori corretti. Non c'è alcun motivo logico, leggendo i requisiti, di testare proprio il numero 99. Il difetto sfugge.
- **White Box (vince):** Avendo accesso al sorgente, il tester vede l'istruzione `if (eta == 99)`. Per soddisfare il criterio della statement coverage (copertura delle istruzioni), è obbligato a scrivere un test case con `eta = 99`. Facendolo, ottiene 1.0 (100% di sconto) e si accorge dell'anomalia. Errore rilevato.

Domanda 2

Discutere lo sviluppo a fasi, mettendo in evidenza analogie e differenze nonché vantaggi e svantaggi rispetto allo sviluppo a cascata.

Risposta

L'idea dello sviluppo a fasi è quella di sviluppare il nostro software in varie release di cui solo l'ultima è completa. Dopo la prima release si procede in parallelo. Gli sviluppatori che lavorano in parallelo preparano la nuova release tenendo conto dei feedback ricevuti dalla release in precedenza.

Vantaggi:

- Il cliente partecipa attivamente allo sviluppo fornendo feedbacks sulle release che sono state pubblicate e sugli aggiornamenti che vengono fatti volta per volta;
- risposta rapida al mercato;
- problemi individuati presto, grazie alla pubblicazione delle release e ai feedback;

Svantaggi:

- costi: rilasciare continuamente software può diventare costoso e pesante
- coinvolgimento del cliente: esso potrebbe non essere disponibile oppure non essere soddisfatto con il rilascio.
- la pianificazione può diventare complessa se i requisiti cambiano da una release all'altra;
- modificare continuamente il prodotto può portare errori alle funzionalità già sviluppate.

L'analogia che osserviamo tra i due modelli consiste nelle fasi che essi attraversano: sia il modello a cascata che il modello iterativo/incrementale seguono le stesse fasi procedurali (requisiti, design...)

La differenza principale tra sviluppo a fasi e sviluppo a cascata consiste nel rilascio: lo sviluppo a cascata prevede un solo rilascio alla fine del processo mentre invece il modello a fasi prevede rilasci continui.

Lo sviluppo a cascata, rispetto al modello iterativo/incrementale, non prevede il coinvolgimento diretto del cliente, di conseguenza può accadere che durante

quell'unico rilascio previsto dal modello il cliente non sia soddisfatto del prodotto e che di conseguenza il lavoro vada ricominciato (nel peggiore dei casi) da capo rallentando il rilascio definitivo. Al contempo, il mancato coinvolgimento del cliente nello sviluppo del prodotto permette una velocità maggiore evitando gli eventuali problemi di disponibilità.

Domanda 3

Illustrare sinteticamente i punti salienti del modello RUP precisando in particolare

a) che differenza c'è fra fasi e discipline

b) quali aspetti sono messi maggiormente in risalto da RUP rispetto al modello a spirale

Risposta

Il RUP (Unified Process) è un meta-modello di processo prescrittivo, orientato agli oggetti, guidato dai casi d'uso e strutturato su tool visuali

a) Differenza fra Fasi e Discipline

- **Le Fasi** (dimensione temporale/orizzontale: Inception, Elaboration, Construction, Transition) indicano quando si svolge un'attività nel ciclo di vita. Ciascuna fase contiene più iterazioni che terminano con una milestone.
- **Le Discipline** (dimensione logica/verticale: Requisiti, Analisi e Design, Codifica, Test) indicano cosa si fa a livello tecnico.
- **Differenza:** A differenza della cascata, le discipline non sono separate nel tempo: in ogni iterazione si lavora su più discipline contemporaneamente, variando solo le proporzioni (es. in Inception si fa molto lavoro di requisiti, in Construction molto codice).

b) Aspetti messi in risalto da RUP rispetto alla Spirale

Pur ereditando l'approccio orientato ai rischi della spirale, il RUP mette maggiormente in risalto:

1. **Forte centratura sull'Architettura:** Impone di definire ed eseguire uno "scheletro" architetturale solido già nella fase di Elaboration per abbattere i rischi strutturali prima di codificare.
2. **Guida tramite Casi d'Uso (Use Case Driven):** I casi d'uso sono il motore centrale che guida ogni fase (requisiti, design, codice e test).
3. **Integrazione di UML:** Adotta sistematicamente UML come standard visuale per modellare e documentare il sistema.

Domanda 4

Descrivere molto brevemente (magari aiutandosi con un diagramma/schema) lo stile architetturale Repository Model

Risposta

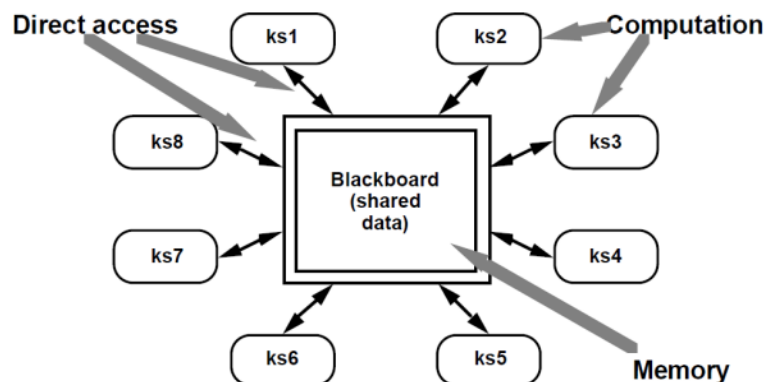
Il repository pattern è uno stile di progettazione ad alto livello in cui i sottosistemi di una stessa applicazione non comunicano direttamente tra loro ma scambiano informazioni con un database centralizzato chiamato blackboard.

Nel repository model tutto la parte computazionale e comunicativa si basa su questa entità centralizzata.

Ad esempio CASE Tools (es. Visual Paradigm): All'interno dello stesso software di modellazione, diversi strumenti lavorano in parallelo leggendo e scrivendo i dati in un unico database di metadati centrale.

Vantaggi: è un modo efficiente di condividere grandi quantità di dati. È possibile avere una gestione centralizzata di backup e security.

Svantaggi: lo schema dei dati è unico, può portare dei problemi in quanto tutti i sottosistemi devono accordarsi su un modello di dati per il repository. Altro problema è l'evoluzione dei dati, è molto costosa. Molto difficile da distribuire.



Domanda 5

Il linguaggio UML può essere utilizzato in modi completamente diversi ad esempio se utilizzato in ambito di programmazione agile oppure se usato in un progetto software safetycritical. In particolare il Fowler nel suo libro presenta tre modi di utilizzo. Quali? Commentare brevemente.

Risposta

I tre diversi modi di utilizzo di UML:

- **Abbozzo:** è la forma più semplice, racchiude le informazioni più importanti del sistema ed aiuta la comunicazione e la discussione.
Tools utilizzati: editor grafici, lavagna
- **Blueprint:** è la forma di UML più precisa e dettagliata. Aiuta effettivamente lo sviluppo progettuale. La codifica è un'attività meccanica.
Tools utilizzati: visual paradigm.
- **Linguaggio di programmazione:** tramite determinate estensioni si può utilizzare UML come linguaggio di programmazione. Serve per fornire un modello eseguibile.
Tools utilizzati: webratio.

Domanda 6

Si consideri il Dott. Rossi che è stato appena nominato quale Project manager del progetto D-Heart che ha come obbiettivo quello di sviluppare un software molto affidabile e sicuro per l'analisi automatica di un elettrocardiogramma. Tale software dovrà essere incorporato nell'elettrocardiografo (cioè il dispositivo hardware munito di sensori che rileva i battiti del cuore del paziente e mostra il grafico) e dovrà analizzare in modo autonomo l'elettrocardiogramma del paziente e segnalare al dottore eventuali anomalie trovate (ad esempio la Sindrome del QT corto). Il software dovrà essere scritto in linguaggio C avendo a disposizione tre sviluppatori con molta esperienza di sviluppo in contesti safety-critical. Dopo avere deciso un modello di processo di sviluppo software adatto al problema e avere commentato la scelta fatta, scegliere un approccio/metodo di testing che vi sembra sensato in questo contesto (commentare anche la scelta dell'approccio di testing proposto).

Risposta

Consapevoli del fatto che stiamo trattando di un sistema delicato da utilizzare in ambito medico, non sono adeguati i metodi agili proprio per la questione della sicurezza/safety.

I modelli perfetti per questo sviluppo si rivelano essere:

- sia il modello a spirale perché introduce (e di conseguenza, sfruttiamo) il concetto di rischio;
- sia il V-Model grazie all'intensa attività di testing che prevede. Ricordiamo infatti che nel V-Model ogni fase di sviluppo è affiancata dalla sua fase di testing.

Le due tecniche principali di testing sono:

- white box: tecnica di testing che viene fatta quando si ha la conoscenza della struttura interna (codice);
- black box: si adotta quando ancora non si sa com'è stato implementato il sistema.

Siccome l'implementazione del sistema avviene prima possiamo adottare un modello di testing white box caratterizzato da diversi TestCase.

Domanda 7

Si consideri il Dott. Bianchi che è stato appena nominato Project manager del progetto Cont-Web che ha come obiettivo quello di sviluppare un programma di contabilità online che si basa su un'archiviazione cloud e che si presenta sotto forma di una Web app (applicazione web). Si supponga di **avere molto poco tempo** per lo sviluppo e di avere a disposizione **parecchi soldi** per effettuare acquisti o per pagare personale ulteriore. Alla data dell'inizio del progetto si hanno **a disposizione tre sviluppatori Web** (front e back end) giovani e con propensione ad imparare nuove tecnologie e nuovi processi di sviluppo software. Si tenga presente anche che il cliente **non ha specificato requisiti molto rigidi** e che quindi è possibile negoziare con lui. Anche lato interfaccia grafica il cliente non è molto esigente. Dopo avere deciso un modello di processo di sviluppo software adatto al problema (ed eventualmente anche i framework/tool di sviluppo) e avere commentato la scelta fatta, scegliere un approccio/metodo di testing che vi sembra sensato in questo contesto (commentare anche la scelta dell'approccio di testing proposto).

Risposta

Il metodo di processo di sviluppo che proporrei per questa situazione è un metodo agile, tenendo conto che abbiamo poco tempo, tanti soldi, requisiti mutabili (non rigidi) e momentaneamente solo 3 programmatori.

In particolar modo suggerirei Extreme Programming tenendo presente che:

- i requisiti non sono rigidi;
- con 3 programmatori possiamo adottare la tecnica di pair programming;
- Possiamo fare una sessione di quick design per capire come strutturare il nostro software ricordando però che deve essere un abbozzo e non il design “solido”, “finale”;
- Inoltre il customer è on site, quindi lo possiamo coinvolgere direttamente nello sviluppo del progetto.

Nella fase di testing scelgo di usare il **TDD** (test driven development) dove, per ogni nuova funzionalità, si aggiunge un singolo caso di test.

Appena il codice supera i casi di test si applica il refactoring.

Ricordiamo altresì che uno dei principi dell’extreme programming è “once and only once”, dunque, evitare i cloni.

Infine, sono previsti gli acceptance testing che hanno l’obiettivo di testare le users stories e sono indice di progresso dello sviluppo del sistema.

Domanda 8

Come spiegato a lezione, e come scritto nel libro UML distilled di Martin Fowler, il linguaggio di modellazione unificato UML può essere usato in contesti di sviluppo del SW diversi (ad esempio sviluppo agile e sviluppo plan-driven) e in modi diversi, ovvero in modo “light” ma anche in modo molto più rigoroso. Dopo avere descritto i tre utilizzi possibili di UML, così come spiegati nel libro UML distilled, in particolare mettendo in risalto le differenze tra loro, vi viene richiesto di associare ad ogni utilizzo un possibile ambito (processo di sviluppo adottato, numero di sviluppatori nel team, dominio applicativo, etc.) ed applicazione (si pensi a tipologie diverse di applicazioni e sistemi SW) che si prestano particolarmente bene. Occorre motivare l’associazione.

Risposta

I tre diversi modi di utilizzo di UML:

- **Abbozzo:** è la forma più semplice, racchiude le informazioni più importanti del sistema ed aiuta la comunicazione e la discussione.

Tools utilizzati: editor grafici, lavagna

L’abbozzo può essere utile nei metodi agili perché indicato per un piccolo team di sviluppatori (4-20 persone). Una sua applicazione può essere un sistema software piccolo e senza particolari esigenze.

- **Blueprint:** è la forma di UML più precisa e dettagliata. Aiuta effettivamente lo sviluppo progettuale. La codifica è un’attività meccanica.

Tools utilizzati: visual paradigm.

Questa tipologia di UML si usa per progetti rigorosi e safety critical(plan driven).

- **Linguaggio di programmazione:** tramite determinate estensioni si può utilizzare UML come linguaggio di programmazione. Serve per fornire un modello eseguibile.

Tools utilizzati: webratio.

Domanda 9

Spiegare la differenza tra il processo di sviluppo tradizionale, nel quale la codifica del codice è eseguita dallo sviluppatore in un linguaggio di programmazione (es. Java), con lo sviluppo denominato MDD (Model Driven Development). Cercare di mettere in luce i pro (ma anche gli eventuali contro) del MDD. Spiegare in quale contesto di sviluppo e per quali applicazioni SW il MDD può risultare superiore rispetto allo sviluppo tradizionale e perchè. Nominare e descrivere un tool MDD se ne avete visto o utilizzato uno.

Risposta

- **Sviluppo Tradizionale:** Lo sviluppatore analizza i requisiti e progetta il sistema (es. tramite diagrammi UML usati come sketch o blueprint), per poi tradurre manualmente queste specifiche scrivendo righe di codice sorgente in un linguaggio di programmazione (es. Java o C++). La codifica è quindi un'attività manuale e spesso creativa.
- **Model-Driven Development (MDD):** La fase di scrittura manuale del codice viene abolita. L'idea cardine è che si esegue direttamente il modello di design oppure il codice dell'applicazione viene generato in modo del tutto automatico a partire dal modello stesso

Pro e Contro del MDD

Vantaggi:

1. **Aumento della Produttività:** Riduce drasticamente i tempi di sviluppo (Time-to-Market) eliminando il collo di bottiglia della scrittura manuale di codice ripetitivo.
2. **Qualità del Codice:** Poiché il codice è generato automaticamente da un motore software standardizzato, si azzerano i bug sintattici e gli errori di battitura tipici dei programmatori umani.

3. **Astrazione e Focus sul Business:** Permette agli sviluppatori (e agli analisti di business) di concentrarsi sulla logica dell'applicazione e sui requisiti, senza perdersi nei dettagli implementativi di basso livello.
4. **Indipendenza dalla Piattaforma:** Lo stesso modello logico (Platform Independent Model) può essere riutilizzato per generare applicazioni in tecnologie diverse (es. Java, .NET) semplicemente cambiando il motore di generazione del tool.

Svantaggi:

1. **Rigore Semantico Richiesto:** Poiché il modello deve essere compilato per generare codice funzionante, i diagrammi non possono contenere alcuna ambiguità o errore sintattico (richiede molto sforzo di modellazione formale).
2. **Costi dei Tool:** i tool di modellazione ed esecuzione MDD industriali richiedono investimenti economici notevoli in termini di licenze software (spesso molto costose).
3. **Rigidità del Codice Generato:** Se l'applicazione richiede comportamenti altamente personalizzati o fuori dagli standard, forzare il tool MDD a generarli (o integrare codice scritto a mano) può risultare estremamente complesso.

Il MDD risulta nettamente superiore rispetto allo sviluppo tradizionale in contesti di sviluppo rapido con requisiti di business volatili. Un esempio di MDD è WebRatio.

Domanda 10

Dopo avere spiegato/descritto cosa è un design pattern (è richiesta la definizione vista a lezione) e perché è utile usare i design pattern durante lo sviluppo di un sistema software sceglierne uno, tra quelli trattati a lezione, e descriverlo nel dettaglio utilizzando il template costituito da: il nome, il problema, la soluzione e le conseguenze nell'adottarlo. E' possibile utilizzare diagrammi UML e fare esempi.

Risposta

Un design pattern è una soluzione elegante ad un problema specifico di design/Object Oriented che costituisce un'unità di riuso. Sono totalmente indipendenti da linguaggi di programmazione OO.

Non sono le prime soluzioni che ci vengono in mente.

Sono utili perché ci permettono di applicare i concetti di progettazione OO come

ad esempio “alta coesione, basso accoppiamento”. Aiutano a creare un buon design e facilitano la comunicazione tra sviluppatori.

Uno di questi design pattern è l’adapter.

Nome: Adapter

Problema che risolve: converte l’interfaccia di una classe in un’interfaccia che il cliente vuole/si aspetta, permettendo alle classi di lavorare assieme.

Soluzione: uso di questo design pattern per adattare l’interfaccia alla richiesta.

Conseguenze: consente a classi con interfacce totalmente differenti e incompatibili di collaborare tra loro senza doverne modificare il codice sorgente originale.

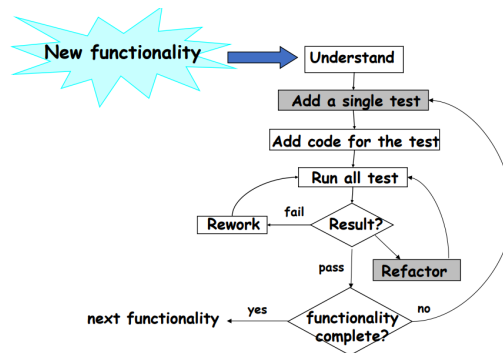
Domanda 11

Dopo avere spiegato la differenza tra Testing manuale e Testing automatizzato del software (pensare al framework di testing JUnit usato in laboratorio) elencate pro e contro dei due approcci. Infine, descrivete come il Testing automatizzato può essere utilizzato in un contesto di sviluppo agile (pensare ad Extreme Programming e/o DevOps). E’ possibile fare esempi con porzioni di codice ed usare diagrammi/schemi per argomentare la risposta.

Risposta

- Nel testing manuale è lo sviluppatore che deve andare a testare le funzionalità del sistema, decidendo chi isolare e cosa verificare. Quest’attività è, per certi versi più precisa ma più lenta.
- Nel testing automatizzato i tester implementano dei **test script** (porzioni di codice in grado di interagire con il SUT, per esempio inserendo input) che sono eseguiti da un framework che dopo la loro esecuzione riporta il risultato (pass/fail). Questa metodologia di testing, al contrario di quella citata sopra, è più veloce ma potrebbe rivelarsi meno precisa perché lo sviluppatore non ha il controllo su quello che il sistema vuole testare.

Nell'Extreme Programming (XP), i test di unità automatici guidano lo sviluppo del codice tramite l'approccio Test-Driven Development (TDD)



Domanda 12

Dopo avere spiegato a cosa serve il Testing framework JUnit visto a lezione, scrivere un caso di test JUnit di esempio e tramite questo spiegare:

1) cosa sono le asserzioni 2) a cosa servono le asserzioni e 3) quali varianti di asserzioni offre JUnit.

Risposta

JUnit è un framework di testing per programmi Java progettato per realizzare e automatizzare il testing di unità.

I suoi possibili usi sono:

1. Scrivere casi di test
2. Eseguire casi di test
3. Raggruppare casi di test in “Test suite”

Esempio:

```

// La classe con la logica da testare (SUT)
public class Calcolatore {
    public static boolean isEven(int number) {
        return number % 2 == 0;
    }
}

@Test
public void testIsEvenWithEvenNumber() {
    int input = 4;
    boolean risultato = Calcolatore.isEven(input);

    // 3. ASSERZIONE (Verifica del risultato atteso)
    assertTrue(risultato);
}
}

```

- 1) affermazione che può essere vera o falsa, automaticamente verificata
- 2) le asserzioni servono per verificare che l'output della funzione testata sia uguale all'output atteso.
- 3)
 - assertEquals
 - assertNotEquals
 - assertNull
 - assertNotNull
 - assertTrue
 - assertFalse
 - assertEquals

Domanda 13

Spiegare cos'è, e a cosa serve, eventualmente aiutandosi con diagrammi UML ed esempi di codice, il pattern architetturale MVC (Model-View-Controller).

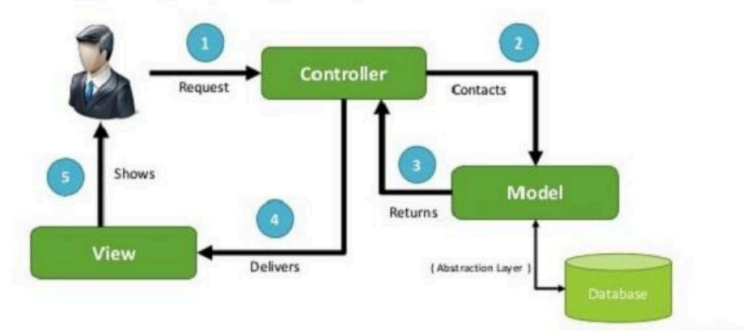
Risposta

Il Model View Controller al contrario dei design pattern definisce l'intera architettura dell'applicazione, mentre i design pattern risolvono problemi "piccoli".

MVC Divide un'applicazione con **GUI** in tre tipologie di componenti

- **model:** abbiamo un modello che contiene i dati e le funzionalità di base, quindi elaborazione dei dati.
- **view:** una o più viste mostrano informazioni all'utente, quindi forniscono l'output.
- **controller:** uno o più controller gestiscono richieste degli utenti ovvero si occupano della gestione di input.

How it works



Il **pattern MVC** è spesso definito un pattern composto perché utilizza l'**observer** per gestire la comunicazione tra i suoi componenti.

Domanda 14

Domanda 15

Dopo avere spiegato i concetti di prospettiva concettuale e prospettiva software in UML e dopo avere specificato in quali fasi dello sviluppo SW queste prospettive possono essere utilizzate, chiarire la differenza tra le due con un esempio.

Risposta

- **Prospettiva concettuale:** va a descrivere “ad alto livello” le entità del dominio di business senza fare riferimenti all’effettiva implementazione (ad esempio: una persona è un’entità fisica, reale). Questa prospettiva è definita e gestita dall’analista.
- **Prospettiva software:** definisce “a basso livello” le scelte implementative relative al dominio di business (ad esempio: una persona è una classe definita da Nome, Cognome...). Questa prospettiva è definita e gestita dallo sviluppatore.

La prospettiva concettuale viene utilizzata principalmente nella fase di Analisi dei requisiti, mentre la prospettiva software viene impiegata nelle successive fasi di Design (progettazione) e Codifica (implementazione).

Esempio:

Vediamo come la stessa entità Libro viene modellata nelle due prospettive:

1. **Prospettiva concettuale:** L'analista disegna una classe per rappresentare il concetto reale del libro:
 - a. **Attributi:** titolo, autore, annoPubblicazione.
 - b. **Operazioni:** nessuna, non esistono metodi
 - c. **Associazioni:** Viene definita un'associazione concettuale di tipo "è preso in prestito da" che collega il Libro al Lettore

Come notiamo gli attributi non hanno un tipo di dato software associato (int, bool ecc).

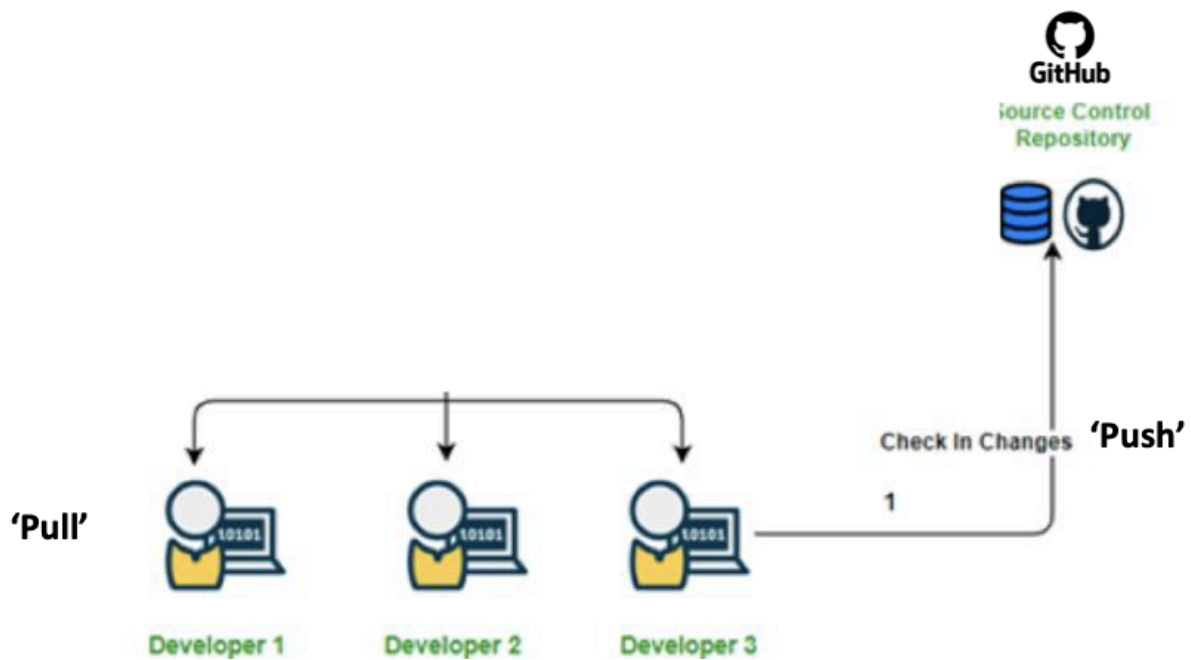
2. **Prospettiva software:** Il progettista traduce il modello concettuale in specifiche pronte per essere codificate.:
 - a. **Attributi:** - titolo : String , - idLibro: int , - disponibilità : boolean
 - b. **Operazioni:** + isDisponibile() : boolean , + prenota(u : Utente) : void
 - c. **Associazioni:** L'associazione concettuale si trasforma in un attributo di riferimento software

Domanda 16

Spiegare, eventualmente facendosi aiutare da uno schema grafico, come avviene il processo di continuous integration (integrazione continua) poi delivery di un software in un'azienda che adotta una metodologia di sviluppo agile. È importante chiarire i vari passi.

Risposta

Pratica di sviluppo software su cui vari sviluppatori integrano il loro lavoro nel **repository** condiviso del progetto. I developer fanno l'operazione di **Pull** e lavorano su alcune funzionalità e una volta finito "**pushano**" il loro lavoro svolto sulla repository in comune. In questo momento ci sono altri software che si accorgono della push e prelevano la nuova codebase ed effettuano la build e mandano in esecuzione vari test e producono una risposta che di solito è una mail mandata a tutti gli sviluppatori.



La Delivery rappresenta l'estensione naturale della CI ed è il ponte verso il rilascio in produzione

- Rilascio in Staging (Replica Production): Una volta superati i test di unità, il server di CD distribuisce automaticamente l'eseguibile (es. un file .jar o un container) in un ambiente di Staging, che è una replica esatta dell'ambiente reale di produzione
- Acceptance Testing: In questo ambiente di staging vengono eseguiti i test di accettazione (funzionali), scritti in collaborazione con il cliente (On-site Customer) sulla base delle User Stories, per verificare che il software risponda esattamente ai requisiti di business concordati
- Pronto per la Produzione: Al termine di questa fase, il sistema produce un incremento di software perfettamente funzionante, testato e pronto per essere rilasciato. Nella Continuous Delivery, il passaggio finale da Staging a Produzione reale (Deploy) avviene tramite una decisione manuale (es. un click di un operatore), mentre se l'intero processo di deploy reale fosse completamente automatizzato si parlerebbe di Continuous Deployment

Domanda 17

Spiegare quali sono e in cosa consistono i principi di codifica nel processo di sviluppo software Extreme Programming (XP)

Risposta

I principali principi di codifica in XP sono:

- Pair programming: caratteristica fondamentale che prevede che la stesura del codice venga fatta da due sviluppatori, uno con più esperienza e uno meno esperto. Solitamente si fa fare coding a quello meno esperto per fare in modo che impari e quello più esperto rimane a sorvegliare, a guidare e a segnalare gli errori;
- Principio KISS (keep it simple stupid): dato che in extreme programming il design viene ricavato direttamente dal codice, quest'ultimo deve rimanere il più semplice possibile;
- Codebase in comune: tutti gli sviluppatori devono avere accesso ad una codebase comune in cui possono accedere e modificare il codice in qualsiasi momento senza alcuna autorizzazione;
- Code Conventions: tutti gli sviluppatori devono seguire le stesse regole sintattiche
- "Think by example" ovvero si parte da un problema concreto, si sviluppa un algoritmo ad hoc e poi si generalizza.

Domanda 18

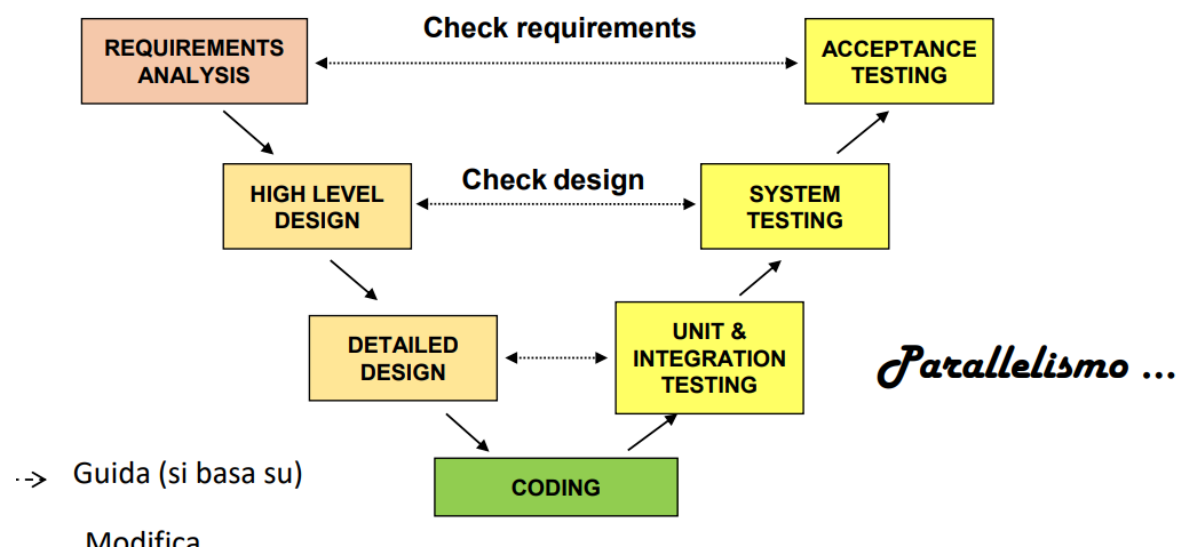
Spiegare, utilizzando uno schema grafico che lo esemplifica, il processo di sviluppo denominato Vmodel. È importante chiarire le varie fasi.

Risposta

Il V-Model è stato il primo modello a introdurre il concetto di “parallelismo”, ovvero ogni fase di sviluppo (a sinistra) ha la sua fase di testing (a destra).

Per ogni fase vengono immediatamente pianificati i test da eseguire (e verranno eseguiti solo dopo che si è conclusa la fase di coding).

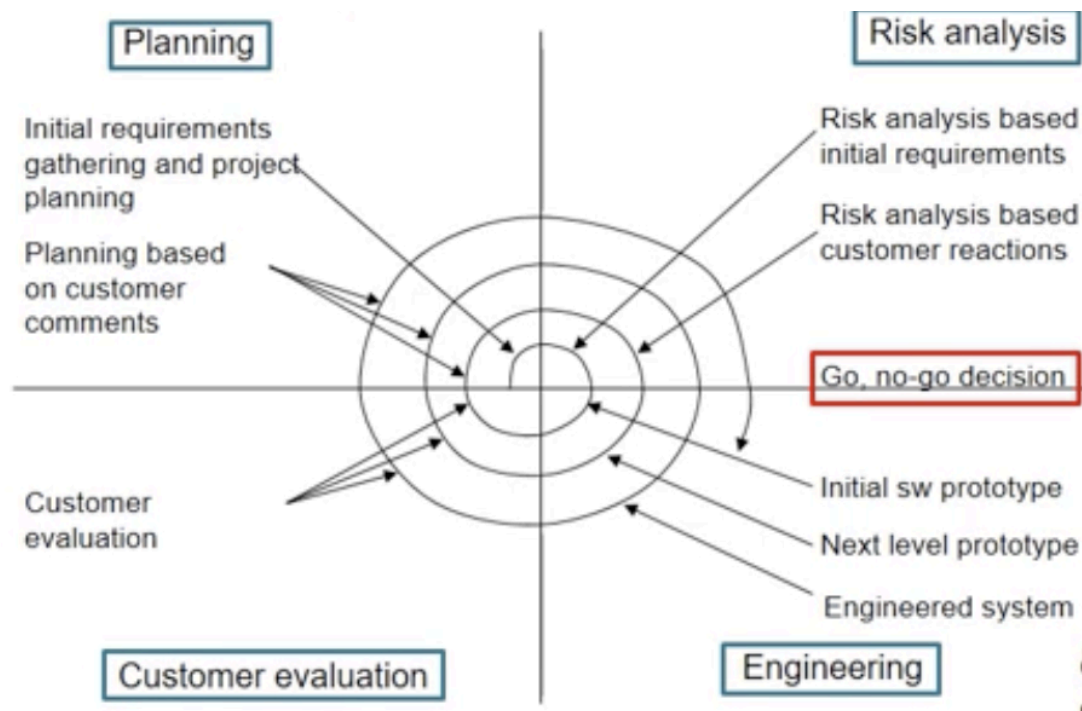
- **Requirements Analysis & Acceptance Testing:** lo scopo è quello di andare a capire cosa il cliente si aspetta, cosa il team deve sviluppare. L'acceptance testing, legato a questa fase, serve per verificare se abbiamo rispettato i requisiti che abbiamo precedentemente catturato e analizzato;
- **High Level Design & System Testing:** qui definiamo l'architettura del nostro software andando a mapparci sopra i requisiti e testando successivamente quello che noi abbiamo creato;
- **Detailed Design & Unit and Integration Testing:** andiamo a definire la struttura in modo dettagliato ad esempio usando un class diagram (definendo, per esempio, come comunicano le classi).
- **Coding:** rappresenta la fase di struttura del codice sorgente che è a metà tra fase di design e fase di test.



Domanda 19

Spiegare, utilizzando uno schema grafico che lo esemplifica, il processo di sviluppo denominato 'Modello a spirale'. È importante chiarire le varie fasi

Risposta



Questo diagramma deve essere letto partendo dal centro. La prima cosa che viene fatta è la parte di **planning**, in questa fase si determinano gli **obiettivi**, le **alternative** e i **vincoli**. Dopo questa fase si passa **all'analisi del rischio** dove si analizzano le alternative e si identificano i rischi ed eventuali soluzioni. La terza fase è **engineering** ed è quella vera e propria dello **sviluppo** del nostro sistema. In questa fase si sceglie un modello di sviluppo ad esempio **waterfall** (nel caso in cui i requisiti siano chiari). La fase conclusiva è quella del **feedback** del nostro cliente. Si continua **a spirale** fino a quando il sistema è completamente finito.

Il vantaggio è che questo modello è adatto per sistemi complessi ed è il primo modello che considera il rischio. Gli **svantaggi** è che non è di certo la soluzione a tutti i problemi, necessita un'opportuna personalizzazione di utilizzo.

Domanda 20

Si supponga di essere il Project manager di un grosso sistema SW legacy in ambito medico del quale non sono documentati i requisiti e non si dispone di documenti di specifica e che il vostro capo vi chieda di garantire la correttezza del sistema. Quali strategie e tecniche di testing (es. white/black box) metteresti in atto? Motivare le scelte e spiegare le strategie/tecniche adottate.

Risposta

- **Fase 1:** effettuare redocumentation per ottenere una vista alternativa del codice ed è anche molto utile per la comprensione; Successivamente facciamo reverse engineering per risalire al design;
- **Fase 2:** strategia di testing black box per testare i requisiti. La problematica è che potrebbero esserci dei “casi nascosti” che non vengono coperti dal black box. Possiamo usare ad esempio la boundary value analysis per testare i confini del sistema e il negative testing per verificare se errori ed eccezioni vengono generati/lanciati correttamente;
- **Fase 3:** eseguiamo white box testing per andare a vedere se esistono dei casi non coperti dal black box.
- **Fase 4:** attuiamo regression testing per verificare che non ci siano stati side effects.

Domanda 21

Spiegare, aiutandosi anche con esempi, la differenza tra:

1. Requisiti SW di Sistema e Utente
2. Requisiti Funzionali e non Funzionali

Risposta

1. Requisiti utente: indicano esattamente quello che vuole il cliente, sono scritti (a volte) con il cliente stesso e in linguaggio naturale. Al contrario i requisiti di sistema indicano rigorosamente “cosa” il sistema dovrà fare e possono essere scritti sia in linguaggio naturale che utilizzando linguaggi più formali.
2. I requisiti funzionali descrivono le funzionalità che il sistema dovrà avere, quindi cosa il nostro sistema dovrà fare. I requisiti non funzionali, invece, descrivono il “come” il sistema dovrà fare determinate cose oppure quali tecnologie vengono adottate.

Domanda 22

Il Requirement Engineering (RE) è il termine usato per descrivere le attività necessarie per raccogliere, documentare e tenere aggiornato l'insieme dei requisiti di un sistema software. Tale processo è un processo di solito iterativo. Spiegare tale processo brevemente eventualmente aiutandosi con un diagramma/schema

Risposta

Lo scopo primario è creare e mantenere aggiornato un documento dei requisiti, che descriva in modo chiaro le funzionalità e i servizi che il sistema dovrà offrire. È un processo iterativo ed è diviso in due parti: sviluppo e gestione.

Abbiamo quattro fasi per lo sviluppo dei requisiti:

1. Elicitation: andiamo a catturare i bisogni del cliente;
2. Requirements Analysis: analisi dei bisogni del cliente per stabilirne fattibilità e correttezza;
3. Requirements Specification: È l'attività di stesura vera e propria dei documenti. I documenti possono essere 2:
 - Definizione(dei requisiti): contiene i requisiti utente, costruisce un contratto fra le parti
 - Specifica(dei requisiti): contiene requisiti di sistema, costituisce starting point per la fase di design;
4. Requirements validation: è una fase di ulteriore analisi ma non viene fatta dallo stesso analista ma di solito viene fatta da altre persone (formal peer reviews) non coinvolte dal progetto stesso

Dopo la fase di cattura ed analisi dei requisiti vi è la fase di Requirements Management, dove avviene la gestione dei requisiti nelle fasi successive allo sviluppo. Attività:

- Approvazione di alcune richieste di cambio dei requisiti
- Implementazione dei cambi richiesti.
- Negoziazione con il cliente

Viene effettuata anche una Impact analysis per i cambi richiesti (se cambio il requisito xyz quali altri requisiti dovrò modificare?).

Domanda 23

Spiegare, anche facendo degli esempi, quali sono le proprietà che caratterizzano (cioè che devono avere) dei 'buoni' requisiti Software.

Risposta

- Correttezza: sono veramente i requisiti voluti?
- Consistenza: non devono esserci requisiti contraddittori;
- Completezza: tutti gli aspetti che il cliente vuole sono coperti nei requisiti.(in assoluto è impossibile);
- Realismo: non si chiede l'impossibile;
- Inequivocabilità: ogni requisito dovrebbe avere solo un'interpretazione;
- Verificabilità: i requisiti vanno espressi in modo che siano testabili
- Tracciabilità: guardando il sistema SW completo dobbiamo essere in grado di risalire al requisito

Domanda 24

Spiegare, quali sono i compiti di un Analista Software all'interno di un'azienda (ovvero di cosa si occupa) e quali sono le qualità e competenze che deve possedere una persona per svolgere questo tipo di lavoro.

Risposta

L'analista software è la persona che:

- si occupa della cattura dei requisiti facendo interviste ai clienti;
- Verifica che le buone proprietà dei requisiti siano verificate;
- predispone il documento dei requisiti;
- Collabora con architetti e progettisti per chiarire requisiti complessi;
- Segue l'evoluzione dei requisiti durante il progetto;

Le competenze che deve avere sono:

- capacità di comunicazione;
- capacità analitica e di problem solving;
- attenzione al dettaglio per evitare ambiguità nei requisiti;
- capacità di negoziazione;
- familiarità con UML;
- familiarità con strumenti di tracciamento dei requisiti

Domanda 25

Dopo avere elencato i principi su cui si basa una metodologia agile, presenti nel Manifesto Agile, descriverli brevemente

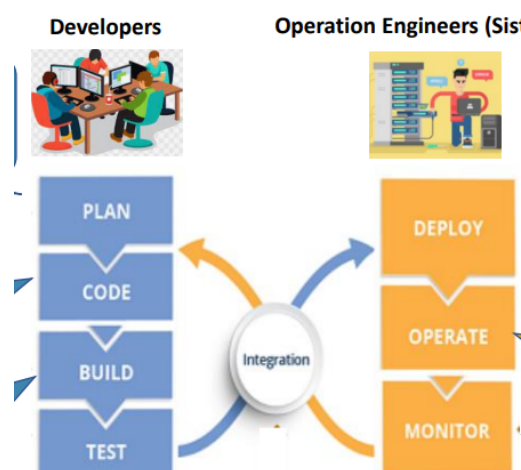
Risposta

1. **Comunicazione tra sviluppatori piuttosto che utilizzo di processi e tools:** per gli sviluppatori che seguono un approccio agile la programmazione è arte e in quanto tale non programmano per il riuso;
2. **Codebase funzionante piuttosto che avere una documentazione completa:** lo scopo principale è sviluppare software funzionante piuttosto che usare risorse per redigere documentazione;
3. **Collaborazione con il cliente piuttosto che negoziare un contratto:** l'obiettivo è quello di soddisfare il cliente senza negoziare mettendo in evidenza i suoi bisogni;
4. **Rispondere al cambiamento piuttosto che seguire un piano:** nei metodi agili bisogna essere flessibili e rispondere velocemente ai cambiamenti, cosa che nei metodi plan driven non si poteva fare a causa della rigidità.

Domanda 26

Rappresentare con una disegno/schema il modello di sviluppo Software moderno visto a lezione basato su Continuous integration e Continuous delivery (DevOps). Descrivere brevemente il suo funzionamento.

Risposta



DevOps è un processo di sviluppo moderno che ha l'obiettivo di mettere in relazione due figure tipicamente in contrasto, ovvero sviluppatori e sistemisti.

1. Le 4 fasi per gli sviluppatori sono:

2. plan: analisi dei requisiti e design;
3. code: stesura codice sorgente. Si usa git per mandare codice su server condiviso;
4. build: si crea codice eseguibile;
5. test: si svolge attività di testing per verificare la qualità dell'implementazione.

Per i sistemisti le fasi sono 3:

1. Deploy: il codice che supera i test viene inserito nel sistema di produzione;
2. Operate: il team operation si assicura che l'infrastruttura sia stabile e che il servizio sia sempre disponibile;
3. Monitor: si raccolgono dati sulle prestazioni dell'applicazione e sull'esperienza utente.

Domanda 27

Dopo avere elencato quali sono i diagrammi UML 2.0 strutturali (Structure Diagram) visti a lezione selezionarne due e spiegare brevemente in quale fase di sviluppo SW possono essere impiegati e per quale motivo

Risposta

Descrivono gli aspetti statici del sistema, cioè le entità che lo compongono e le relazioni logiche o fisiche tra di esse.

1. **Class Diagram** (Diagramma delle Classi)
2. **Object Diagram** (Diagramma degli Oggetti)
3. **Component Diagram** (Diagramma dei Componenti)
4. **Package Diagram** (Diagramma dei Package)
5. **Deployment Diagram** (Diagramma di Dispiegamento)

Il class diagram è sicuramente uno dei diagrammi UML più diffusi. Esso serve per descrivere le classi di un sistema software e le relazioni tra di esse. Nella descrizione delle classi essi vogliono mettere in evidenza nome, eventuali attributi ed eventuali metodi.

Abbiamo due tipi di prospettiva:

- **prospettiva software:** gli elementi rappresentati sono entità proprie del sistema software;
- **prospettiva concettuale:** gli elementi descritti sono entità “fisiche” del dominio di business e del mondo reale.

I class diagrams si usano all'interno della fase di design del software con l'obiettivo di semplificare la vista delle classi.

Gli object diagrams sono direttamente collegati ai class perché il loro scopo è quello di semplificare la comprensione dei class diagrams complessi.

Domanda 28

Dopo avere elencato quali sono i diagrammi UML 2.0 'di comportamento' (Behavior Diagram) visti a lezione selezionarne due e spiegare brevemente in quale fase di sviluppo SW possono essere impiegati e per quale motivo

Risposta

Descrivono gli aspetti dinamici del sistema, ossia come il software si comporta, come evolvono i suoi stati nel tempo e come gli elementi collaborano e reagiscono alle sollecitazioni esterne.

Si dividono a loro volta in due sottogruppi:

- **Comportamento generale**
 - Use case Diagram
 - Activity Diagram
 - State Machine Diagram
- **Diagrammi di interazione**
 - Sequence Diagram
 - Communication Diagram

Lo scopo principale dello state machine diagram è quello di mostrare come un'entità cambia il suo comportamento a seconda dello scaturirsi di un evento esterno. Questi sono composti da nodi (che rappresentano gli stati, solitamente dei rettangoli con angoli smussati) e transizioni (rappresentate come delle linee tra i nodi). Le transizioni hanno la particolarità di poterle indicare tramite un evento oppure con una guardia.

Lo state machine diagram viene usato nella fase di low level design per modellare il comportamento dinamico di un singolo oggetto.

Un altro diagramma di comportamento molto importante è l'activity diagramma. Quest'ultimo molto simile ai diagrammi di flusso, infatti tendono a voler rappresentare una procedura algoritmica.

Domanda 29

Nel contesto del software testing viene spesso usato/nominato il concetto di copertura (coverage).

Spiegare in che cosa consiste questo concetto e perché è così importante aiutandosi con esempi.

Risposta

Il concetto di copertura viene introdotto dalla metodologia di testing White box dove nella quale abbiamo l'accesso al codice sorgente. L'idea della coverage è, come dice il nome, quella di verificare quanto codice viene "coperto" ovvero quante porzioni di codice sono state effettivamente controllate. La coverage viene misurata in percentuale, l'obiettivo logico sarebbe avere una coverage del 100% ma ovviamente questo non è sempre possibile (All Path coverage). Ci sono diversi motivi per il quale è impossibile raggiungere SEMPRE una percentuale simile:

- Cammini infesabile ovvero alcuni cammini non possono essere raggiunti perché la condizione per percorrere quei cammini è impossibile (ad esempio una condizione logica del tipo $n > 0 \ \&\& \ n < 0$)
- Presenza di cicli
- Input domain elevato, trovare tutti i possibili input a livello probabilistico è impossibile.

La coverage rimane importante in quanto ci aiuta nello sviluppo del testing, può essere molto utile soprattutto nello Unit testing ed è indice di progresso. Lo unit Testing è semplicemente una forma di testing dove possiamo testare delle singole unità.

Alcuni esempi di Coverage sono:

1. **Node statement coverage:** in questo caso è possibile ottenere una coverage del 100% in quanto andiamo a testare quanti nodi vengono coperti
2. **Branch coverage:** implica node statement coverage, l'obiettivo sarebbe quello di raggiungere ogni singolo branch, ma come detto prima non sempre è possibile (cammini infesbale)
3. **Multiple condition Coverage:** implica branch e node coverage, l'idea sarebbe testare tutte le condizioni atomiche (& e ||). Qui è possibile raggiungere 100%.

4. **All Path coverage:** impossibile raggiungere 100%, l'idea sarebbe raggiungere tutti i possibili cammini.

Domanda 30

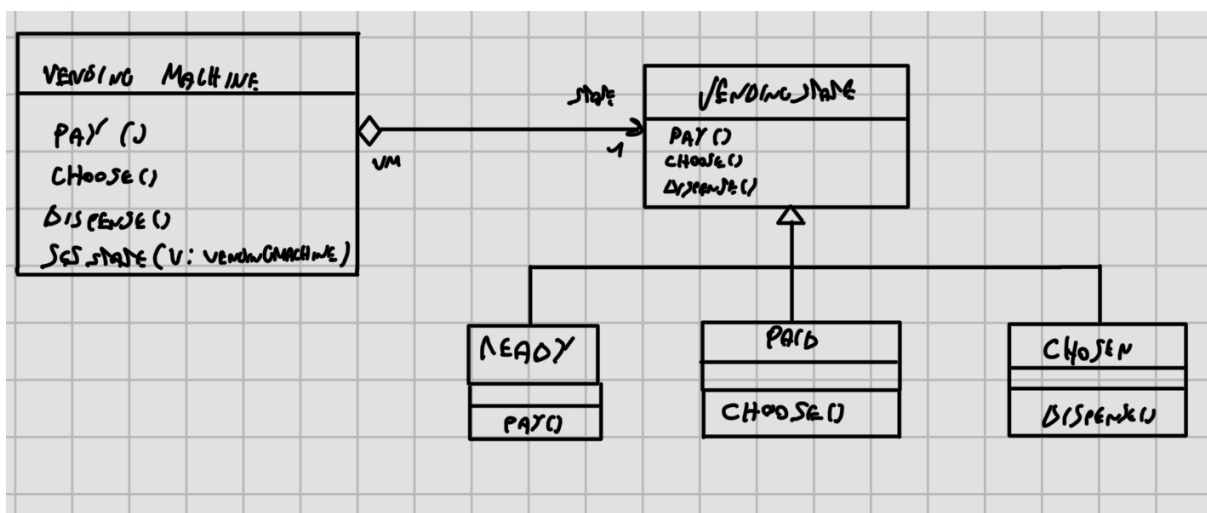
Illustrare il funzionamento del design pattern **State** includendo:

1. Una spiegazione breve ma dettagliata del concetto e del suo utilizzo.
2. Il diagramma delle classi UML, applicato a un esempio concreto, che evidenzi le relazioni tra gli oggetti.

Risposta

- 1) Lo state pattern ha l'obiettivo di implementare (bene) lo state machine diagram in un linguaggio che non lo supporta nativamente, il suo obiettivo quindi è quello di evitare l'uso di una cascata di IF o il classico switch case. La sua alternativa sarebbe quella di realizzare classi concrete per ogni tipo di stato e di metterli in una gerarchia. I vantaggi sono: il comportamento associato ad uno stato dipende solo da una classe, ogni stato è rappresentato da una classe (evitiamo come ho detto prima il grosso caso dello switch), è semplice cambiare comportamento di uno stato. Lo svantaggio è che si incrementa il numero delle classi

2)



Domanda 31

Spiegare che cosa sono e per cosa sono utilizzati gli Object Diagram UML. Fare un esempio di Object Diagram UML a partire da un Class Diagram.

Risposta

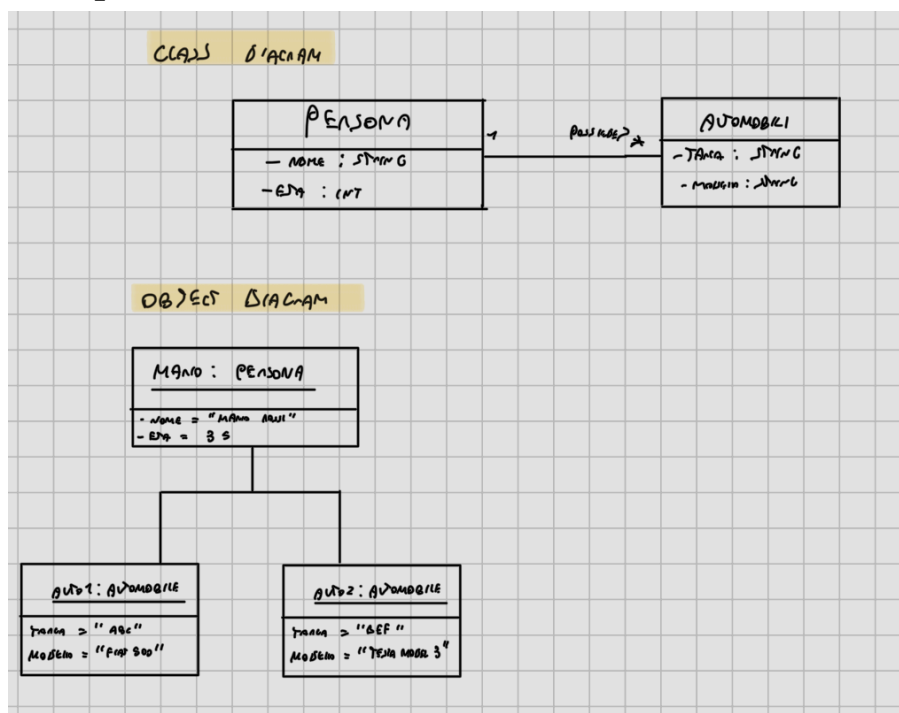
Mentre il Class Diagram mostra l'architettura astratta e generica del sistema (definendo le classi, i loro attributi, le operazioni ammissibili e le regole di associazione), l'Object Diagram mostra le istanze concrete delle classi (gli oggetti veri e propri residenti in memoria) e le istanze delle associazioni (dette link) che li collegano in quel preciso momento.

Le caratteristiche sintattiche fondamentali di un oggetto in UML sono:

- **Identificatore dell'oggetto (Notazione sottolineata)**
- **Compartimento degli attributi:** A differenza del Class Diagram (che indica nome e tipo dell'attributo, es. saldo : double), l'Object Diagram mostra i valori effettivi ed istantanei assunti dagli attributi (es. saldo = 300.00)
- **Assenza di operazioni**
- **Link:** I collegamenti tra oggetti sono linee continue senza molteplicità/cardinalità

Gli object diagram servono a chiarire i class diagram “complessi”.

Esempio



Domanda 32 (generata)

Spiegare il funzionamento dei State Machine Diagram UML (stati, eventi, guardie, azioni entry , exit , do , semantica run-to-completion) e illustrare come una State Machine possa essere implementata in Java (confrontando l'approccio tramite switch e l'uso del Design Pattern State)

Risposta

Lo state machine è un diagramma UML comportamentale con lo scopo di visualizzare i cambiamenti di stato interni di un'entità in base a sollecitazioni esterne ed eventi.

Graficamente le state machine UML sono rappresentate come grafi. In particolare:

- nodo: stato
- arco: transizione

Una macchina a stati è sempre associata ad un'entità (es. classe o sistema SW) e ne descrive il suo comportamento. La macchina a stati riceve eventi che vengono salvati su una coda ed estratti uno alla volta (Buffer). La semantica di questi eventi è di tipo run-to-completion: un evento viene estratto solo dopo che il precedente è stato processato. Se ci sono più transizioni eseguibili in un dato momento solo una viene eseguita in modo non deterministico.

Nella macchina a stati troviamo le transizioni le quali rappresentano come si passa da uno stato ad un altro. Ogni transizione è composta da:

- Evento: un trigger che attiva il passaggio di stato
- Guardia: una condizione che, se vera, permette il passaggio di stato
- Attività: una o più azioni che sono compiute prima di cambiare stato.

Sono tutte e tre opzionali.

All'interno di uno stato si possono usare alcune attività speciali, indicate tramite:

- entry: eseguita quando l'oggetto entra nello stato
- exit: eseguita quando l'oggetto esce dallo stato
- do (do-activity): eseguita mentre l'oggetto è nello stato

Possiamo realizzare una macchina a stati in Java tramite una cascata di if oppure utilizzando lo switch. Il problema principale di questi due riguarda la complessità del codice, ovvero rischiamo di avere un codice troppo complesso da gestire e

leggere. La soluzione migliore sarebbe quella di usare lo State Pattern che ci permette di rappresentare la macchina a stati in modo visuale.

Domanda 33 (generata)

Descrivere i Sequence Diagram UML, chiarificando la differenza tra System Sequence Diagram (SSD) e Sequence Diagram di Design, e spiegando l'uso dei frammenti combinati (alt , loop , opt)

Risposta

I sequence diagram sono diagrammi UML d'interazione con l'obiettivo di mostrarci la sequenza temporale dei messaggi scambiati fra le varie entità. Questi diagrammi sono utilizzati nella fase di design perché ci permettono di avere pure una buona base per la successiva fase di implementazione.

All'interno del sequence diagram è possibile utilizzare i frame combinati che ci aiutano a descrivere determinate situazioni.

- alt: sta per “alternative” e corrisponde al costrutto “if/else”;
- loop: indica un ciclo e corrisponde al ciclo for. Infatti viene indicata una guardia da rispettare;
- opt: sta per “option” e corrisponde all’if (senza else) qualora ci sia una sola condizione, mentre invece se ci fossero più condizioni successive allora sarebbe da considerarsi come switch.

SSD (System Sequence Diagram): illustra gli eventi di input ed output del sistema relativi ad uno scenario (o più) di un caso d'uso. Quindi il suo scopo è quello di stabilire con precisione quali sono gli eventi di input (operazioni di input e parametri) e di output (dati di ritorno) del sistema

Domanda 34 (generata)

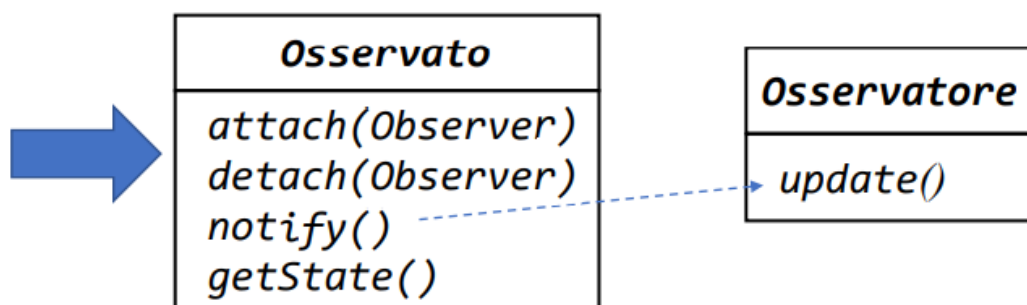
Descrivere il Design Pattern Observer (problema, soluzione, relazioni publish-subscribe, metodi attach / detach / notify / update), mostrando il Class Diagram UML e il Sequence Diagram dei messaggi per illustrare come si ottiene il disaccoppiamento tra Model e View

Risposta

L'observer ha lo scopo di creare un meccanismo dove non appena un oggetto cambia stato, i componenti interessati (gli osservatori) vengono notificati.

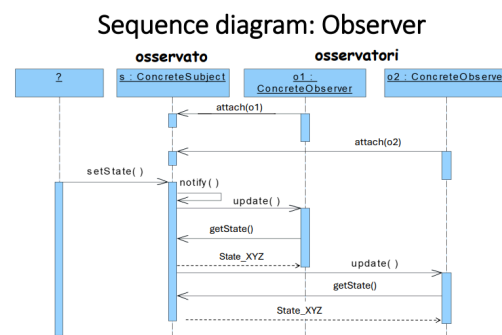
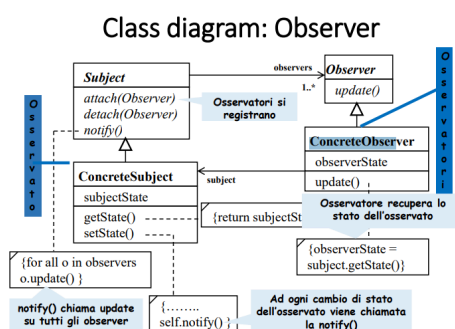
L'idea è quella di avere una lista dinamica di osservatori i quali si registrano all'oggetto osservato.

Possiamo realizzare questo sistema tramite un'interfaccia dove l'osservato ha il metodo `notify()` il quale viene chiamato non appena l'oggetto cambia stato. Inoltre possiede le operazioni `attach(Observer)` e `detach(Observer)` per consentire la registrazione/rimozione dinamica a runtime. `notify()` a sua volta chiama `update()` che si trova all'interno della classe di ciascun osservatore, e ciascun observer può decidere di invocare l'operazione `getState()` (la quale si trova dentro all'oggetto osservato).



Il vantaggio principale è che implementa Broadcast e garantisce disaccoppiamento tra osservatori e osservato (l'osservato sa che esiste questa lista senza conoscerne la struttura concreta). È la base del pattern MVC.

Lo svantaggio è che potrebbe portare a modifiche inaspettate e siccome usa broadcast questo errore viene propagato nel sistema.



Domanda 35 (generata)

Descrivere il Design Pattern Adapter, illustrando la differenza strutturale e implementativa tra Object Adapter (basato su composizione/delega) e Class Adapter (basato su ereditarietà)

Risposta

Il design pattern Adapter è un design pattern strutturale.

Converte l'interfaccia di una classe in un'altra interfaccia che il cliente si aspetta.

Permette a delle classi di lavorare assieme, anche se non potrebbe, visto che hanno interfacce incompatibili.

Abbiamo due modi per implementare l'adapter:

- **Object Adapter:** si basa su delega.

L'idea sarebbe: ObjectAdapter converte, ovvero adatta, l'interfaccia che il cliente si aspetta (Obbiettivo) all'interfaccia dell'adattato. Il cliente usa ObjectAdapter come fosse l'oggetto Adattato. ObjectAdapter possiede il riferimento a Adattato e sa come invocarlo

- **Class Adapter:** si basa su ereditarietà.

Il Class Adapter richiede di ereditare sia dalla classe Adattato sia dalla classe Obiettivo. Poiché Java non supporta l'ereditarietà multipla tra classi, in Java l'Class Adapter è realizzabile solo se l'interfaccia attesa (Target) è un'interfaccia vera e propria e non una class.

Domanda 36 (generata)

Spiegare la differenza tra la relazione <<include>> e la relazione <<extend>> nei Use Case Diagram UML. Illustrare con un diagramma ed un esempio concreto quando un sotto flusso va modellato come inclusione obbligatoria e quando come estensione condizionale con punti di estensione.

Risposta

La relazione <<include>> e la relazione <<extend>> sono fondamentali per la comprensione di uno use case diagram.

- <<include>>: rappresenta un comportamento obbligatorio per la realizzazione di uno use case (ad esempio, nel sistema del bancomat, per effettuare una qualsiasi operazione bisogna inserire PRIMA il codice pin della carta);
- <<extend>>: rappresenta, invece, un comportamento opzionale che molto spesso aggiunge dettagli ad uno use case già presente (ad esempio, se in un sistema per la gestione di un negozio abbiamo lo use case “Ordina cibo”

potremmo rappresentare mediante la relazione <<extend>> lo use case “Ordina vino” tramite un collegamento)

Domanda 37 (generata)

Descrivere il problema dell'Impedance Mismatch (mancata corrispondenza tra il modello a oggetti e il modello relazionale nei database) e spiegare la differenza tra le strategie di recupero dei dati Lazy Loading ed Eager Loading negli ORM, evidenziandone vantaggi e svantaggi prestazionali

Risposta

L'Impedance Mismatch è il divario concettuale e tecnico tra il modello ad oggetti (OO) del codice e il modello relazionale (RDBMS) del database.

- **Modello OO:** Grafo di oggetti caratterizzato da stato, comportamento (metodi), ereditarietà, polimorfismo e incapsulamento.
- **Modello Relazionale:** Tabelle piatte prive di comportamento, composte da tipi di dato semplici e collegate unicamente da chiavi primarie ed estere.
- **Incompatibilità principali:** Gestione dell'ereditarietà (assente nei DB relazionali), granularità dei tipi dati, identità degli oggetti vs chiavi primarie e navigabilità delle associazioni.

Definiscono la strategia utilizzata dagli ORM per scaricare dal database le entità collegate a un oggetto principale:

- **Lazy Loading** (Caricamento Pigro / On-Demand):
 - **Come funziona:** Carica inizialmente solo l'oggetto principale (tramite oggetti proxy); i dati correlati vengono recuperati con query aggiuntive solo quando si accede esplicitamente a quelle proprietà.
 - **Vantaggi:** Molto rapido ed efficiente in termini di memoria ed esecuzione iniziale.
 - **Svantaggi:** Rischio di eccezioni se si accede ai dati a transazione chiusa (LazyInitializationException) e rischio del problema delle N+1 query.
- **Eager Loading** (Caricamento Immediato):
 - **Come funziona:** Carica subito l'oggetto principale e tutte le sue associazioni in un'unica query SQL (sfruttando le JOIN).
 - **Vantaggi:** Risolve il problema delle N+1 query e garantisce che tutti i dati siano subito disponibili in memoria.
 - **Svantaggi:** Può risultare molto pesante se scarica enormi moli di dati collegati che non verranno effettivamente usati.

Domanda 38 (generata)

Definire i concetti di High Cohesion e Low Coupling nella progettazione software. Descrivere la scala dei tipi di accoppiamento (da Content Coupling a Data Coupling) e spiegare perché un accoppiamento elevato rende il codice difficile da mantenere e testare.

Risposta

L'aver un'alta coesione e un basso accoppiamento è uno dei principi di buona progettazione Software (sono criteri per la modularizzazione).

La coesione misura quanto gli elementi contenuti all'interno di un singolo modulo/classe siano strettamente correlati tra loro per esprimere un'unica astrazione. Avere alta coesione significa che la componente è specializzata solo nello svolgere una determinata operazione ed è un vantaggio perché in questo modo siamo sicuri che la classe è più facile da gestire e da mantenere. Avere un basso accoppiamento significa che la classe non dipende troppo da altre classi e favorisce il riuso.

Abbiamo diversi tipi di accoppiamento:

- **Accoppiamento buono:** Un esempio sono le chiamate di funzioni create da altre classi (riuso).
- **Accoppiamento cattivo:** Un esempio è il content coupling dove abbiamo delle istruzioni di tipo GO TO (tipico nei linguaggi legacy).

Domanda 39 (generata)

Spiegare il principio del pattern GRASP Creator, chiarendo quali regole guidano l'assegnazione delle responsabilità di calcolo e creazione tra le classi di un sistema.

Risposta

Il pattern GRASP Creator ha l'obiettivo di rispondere alla domanda "chi crea un oggetto di tipo A?".

La soluzione che propone è quella di assegnare la responsabilità di creazione a una classe B se almeno una di queste condizioni è verificata (più sono vere meglio è):

- B aggrega A
- B usa strettamente A
- B ha i dati per l'inizializzazione di A

Tuttavia esistono casi in cui **non conviene** applicare il pattern Creator, per esempio se volessi nascondere la logica di creazione (entra in gioco factory).

Domanda 40

Quali sono i difetti del modello a cascata e come hanno fatto i modelli evolutivi a risolvere questi difetti

Risposta

I difetti principali del modello a cascata sono:

- **Niente feedback** tra le varie fasi
- **Modello sequenziale**(niente parallelismo)
- **Unica data di consegna**

I modelli evolutivi si basano sull'idea di creare il nostro sistema software in varie release, di cui solo l'ultima e quella definitiva. Rispetto al waterfall dopo la prima release si procede in parallelo e avendo un costante feedback da parte degli utenti si riesce a rispettare i bisogni/requisiti dell'utente.

I modelli evolutivi che abbiamo visto nel nostro corso sono:

1. **Modelli incrementali:** L'idea è che ad ogni release si aggiunge una nuova funzionalità, abbiamo due strategie: O aggiungiamo le funzionalità più a rischio oppure le funzionalità importanti per il cliente.
2. **Modelli iterativi:** L'idea che dalla prima release si trovano tutte le funzionalità del sistema e ad ogni release si vanno semplicemente ad evolvere/migliorare

NOTA: in entrambi i casi abbiamo un numero indefinito di Release.

Domanda 41

Spiega la differenza tra Reverse Engineering e Re-engineering. Fornisci un esempio concreto per ciascuna attività nel contesto di un progetto software reale o ipotetico.

Risposta

Il Reverse Engineering è una tecnica che ci permette di ricavare il design/requisiti a partire dal codice sorgente, di solito nei contesti reali un esempio di Reverse Engineering è risalire al codice sorgente a partire da un eseguibile (es. videogiochi).

Il Re-Engineering mette insieme diverse tecniche insieme, più precisamente comprende:

- Una fase di Reverse Engineering per ricavare la documentazione/risalire ai requisiti

- Una fase di Forward Engineering nel quale si sfrutta semplicemente il normale sviluppo di un software
- Una fase di modifiche e specifiche del design

Un esempio di Re-engineering è il porting da RPG a C++